VULNERABILITY ASSESSMENT REPORT

Target:	`demo.testfire.net` (Altoro Mutual Demo Banking Application)
IP Address:	65.61.137.117
Prepared by:	Sandile Theron Lubisi
Contact:	072 873 2592
Date:	25 May 2026
Internship:	Future Interns — Cyber Security Track (CS)
Tools Used:	Nmap 7.98 | OWASP ZAP 2.17.0 | Chrome DevTools

EXECUTIVE SUMMARY

A black-box vulnerability assessment was conducted on the Altoro Mutual demo web application hosted at `demo.testfire.net` (IP: 65.61.137.117). The assessment was performed using industry-standard tools including Nmap 7.98, OWASP ZAP 2.17.0, and manual browser inspection via Chrome DevTools.

The assessment identified 14 vulnerabilities across multiple risk categories. The assessment confirmed a Critical-rated SQL Injection flaw through manual testing, alongside multiple High, Medium, and Low severity findings including a Denial-of-Service exposure, weak cryptographic configurations, and missing security headers.

Key findings include:

- SQL Injection confirmed on the login form (Manual Test)
- Slowloris Denial-of-Service attack exposure (CVE-2007-6750)
- Weak Diffie-Hellman TLS key exchange (1024-bit)
- Missing Content Security Policy across all pages
- Missing Anti-Clickjacking header across all pages
- CSRF tokens absent on login, feedback, and subscription forms
- Server version information disclosed in HTTP response headers

Immediate remediation is recommended for the SQL Injection and Slowloris DoS findings, followed by the implementation of all missing HTTP security headers to protect customer interactions.

SCOPE AND METHODOLOGY

Target Information

- **Target URL:** `http://demo.testfire.net`
- **IP Address:** 65.61.137.117
- **Application:** Altoro Mutual — Demo Banking Web Application
- **Server:** Apache Tomcat / Coyote JSP Engine 1.1

Assessment Type

Black-box, non-destructive assessment. No credentials were used during scanning phases. Manual testing used only publicly accessible pages.

Tools Used

- **Nmap 7.98** — Port scanning, service detection, vulnerability scripts
- **OWASP ZAP 2.17.0** — Automated passive web vulnerability scanning
- **Chrome DevTools** — Manual HTTP header, cookie, and source code inspection

Scan Types Performed

1. Service Version Scan (`nmap -sV`)
2. Aggressive Scan (`nmap -A`)
3. Full Port Scan (`nmap -p-`)
4. Vulnerability Script Scan (`nmap --script vuln`)
5. ZAP Baseline Passive Scan
6. Manual browser inspection

Assessment Date:19–25 May 2026

FINDINGS SUMMARY TABLE

#	Vulnerability	Tool	Risk	OWASP
1	SQL Injection — Login Form	Manual Test	CRITICAL	A03: Injection
2	Slowloris DoS (CVE-2007-6750)	Nmap	HIGH	A05: Misconfiguration
3	Weak DH Key Exchange (1024-bit)	Nmap	HIGH	A02: Crypto Failures
4	Absence of Anti-CSRF Tokens	ZAP	MEDIUM	A01: Broken Access Control
5	Content Security Policy Not Set	ZAP + DevTools	MEDIUM	A05: Misconfiguration
6	Missing Anti-Clickjacking Header	ZAP + DevTools	MEDIUM	A05: Misconfiguration
7	Source Code Disclosure (SQL)	ZAP	MEDIUM	A05: Misconfiguration
8	Subresource Integrity Missing	ZAP	MEDIUM	A08: Software Integrity

#	Vulnerability	Tool	Risk	OWASP
9	Cookie Missing Security Attributes	ZAP + DevTools	MEDIUM	A07: Auth Failures
10	Cross-Domain JS File Inclusion	ZAP	LOW	A05: Misconfiguration
11	Server Version Disclosed	Nmap + DevTools	LOW	A05: Misconfiguration
12	X-Content-Type-Options Missing	ZAP + DevTools	LOW	A05: Misconfiguration
13	HTML Comments in Source Code	DevTools	LOW	A05: Misconfiguration
14	Quirks Mode / Outdated DOCTYPE	DevTools	LOW	A05: Misconfiguration

INDIVIDUAL FINDING DESCRIPTIONS

1: SQL Injection — Login Form

Risk Level:	🔴 CRITICAL
Discovered By:	Manual Testing (Chrome Browser)
Evidence Page:	Login_SQL Injection Manual Test.png
OWASP:	A03:2021 — Injection

WHAT IS IT?

SQL Injection occurs when an attacker inserts malicious database commands into an input field. The application passes this input directly to the database without checking it first, causing the database to execute unintended commands instead of just reading data.

WHAT WAS FOUND?

When the username field on the login page was given the input `admin'` (with a single quote), the application returned a raw database error: *"Syntax error: Encountered 'test' at line 1, column 67."* This confirms the input was passed directly to the database without sanitization.

BUSINESS IMPACT

An attacker could bypass the login page entirely without a valid password. All customer bank accounts, balances, and personal information could be accessed, modified, or deleted. This would result in severe regulatory penalties under South Africa's POPIA (Protection of Personal Information Act).

HOW TO FIX IT

- Use parameterized queries (prepared statements) for all database interactions — never insert user input directly into SQL strings.
- Validate and sanitize all user inputs on the server side.
- Display generic error messages to users — never expose database errors publicly.

2: Slowloris Denial-of-Service Attack

Risk Level:	🔴 HIGH
Discovered By:	Nmap (--script vuln)
Affected Ports:	443, 8080
OWASP:	A05:2021 — Security Misconfiguration

WHAT IS IT?

Slowloris is a Denial-of-Service attack that opens many partial HTTP connections to a web server and holds them open as long as possible. By sending partial HTTP requests very slowly, it exhausts the server's connection pool, preventing real users from accessing the site.

WHAT WAS FOUND?

Nmap's vulnerability script confirmed the server on ports 443 and 8080 is LIKELY VULNERABLE to Slowloris (CVE-2007-6750). The server does not appear to have connection timeout limits configured to defend against this attack.

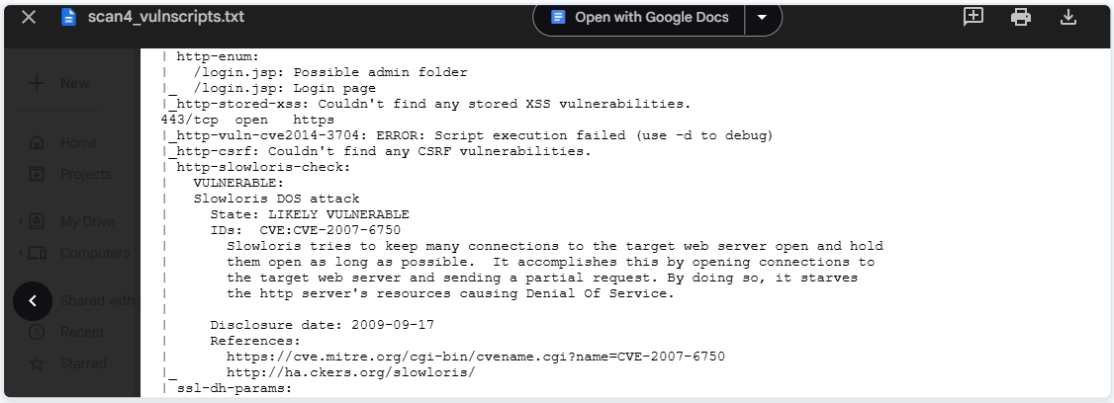
BUSINESS IMPACT

An attacker could take the banking website completely offline with minimal resources — a single laptop could bring down the entire service. Customers would be unable to access their accounts, causing serious reputational and financial damage.

HOW TO FIX IT

- Configure the web server to limit the number of simultaneous connections per IP address.
- Set aggressive timeout values for incomplete HTTP requests.

- Deploy a Web Application Firewall (WAF) or reverse proxy (e.g., Cloudflare) that handles connection management.



3: Weak Diffie-Hellman (DH) Key Exchange — 1024-bit

Risk Level:	🔴 HIGH
Discovered By:	Nmap (--script vuln) on port 443
OWASP:	A02:2021 — Cryptographic Failures

WHAT IS IT?

When your browser connects to a secure (HTTPS) website, both sides agree on an encryption key using a mathematical process called Diffie-Hellman key exchange. A 1024-bit key size is considered too weak by modern standards and can potentially be broken by eavesdropping.

WHAT WAS FOUND?

The TLS configuration on port 443 uses RFC2409/Oakley Group 2 with a 1024-bit modulus. The outdated cipher suite `TLS_DHE_RSA_WITH_AES_256_CBC_SHA` was detected.

BUSINESS IMPACT

A sophisticated attacker could passively intercept encrypted traffic between customers and the bank and eventually decrypt it. Sensitive data such as login credentials, account numbers, and transaction details could be exposed.

HOW TO FIX IT

- Upgrade to a minimum 2048-bit DH group, preferably 4096-bit.
- Use modern cipher suites — prefer ECDHE (Elliptic Curve Diffie-Hellman) over standard DHE.
- Disable all cipher suites using weak DH parameters.

4: Absence of Anti-CSRF Tokens

Risk Level:	🟠 MEDIUM
Discovered By:	Nmap + OWASP ZAP
Affected Pages:	login.jsp, feedback.jsp, subscribe.jsp
OWASP:	A01:2021 — Broken Access Control

WHAT IS IT?

Cross-Site Request Forgery (CSRF) tricks a logged-in user into unknowingly submitting a form action on a website. Anti-CSRF tokens are secret, unpredictable values in forms that verify the form was intentionally submitted by the real user from the correct page.

WHAT WAS FOUND?

ZAP and Nmap both confirmed that three key forms — the login form, the feedback form, and the subscription form — contain no Anti-CSRF tokens to validate user intent.

BUSINESS IMPACT

An attacker could trick a logged-in customer into unknowingly performing bank actions (e.g., submitting a fund transfer form) by getting them to visit a malicious webpage. The customer would not know anything happened.

HOW TO FIX IT

- Generate a unique, unpredictable token for every form and include it as a hidden field.
- Validate the token on the server side before processing any form submission.
- Apply SameSite attributes to all session cookies (See Finding #9).

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Absence of Anti-CSRF Tokens	Medium	3
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Missing Anti-clickjacking Header	Medium	Systemic
Source Code Disclosure - SQL	Medium	1
Sub Resource Integrity Attribute Missing	Medium	1
Cookie without SameSite Attribute	Low	4
Cross-Domain JavaScript Source File Inclusion	Low	1
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	3
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	3
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	3
Dangerous JS Functions	Low	1
Permissions Policy Header Not Set	Low	Systemic
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	Systemic
X-Content-Type-Options Header Missing	Low	Systemic
Information Disclosure - Suspicious Comments	Informational	6
Modern Web Application	Informational	4
Session Management Response Identified	Informational	4
Storable and Cacheable Content	Informational	Systemic

5: Content Security Policy (CSP) Header Not Set

Risk Level:	🟠 MEDIUM
Discovered By:	OWASP ZAP + Chrome DevTools
Affected:	All pages (Systemic)
OWASP:	A05:2021 — Security Misconfiguration

WHAT IS IT?

A Content Security Policy is an HTTP header that tells the browser exactly which sources of scripts, styles, images, and other content are trusted. Without it, the browser will load and execute content from any source — including malicious ones.

WHAT WAS FOUND?

The Response Headers screenshot from DevTools confirms that no `Content-Security-Policy` header is present in any server response. ZAP flagged this as Systemic.

BUSINESS IMPACT

Without CSP, the site is significantly more vulnerable to Cross-Site Scripting (XSS) attacks. An attacker could inject malicious JavaScript that steals session tokens, redirects users to phishing pages, or silently sends customer data to an external server.

HOW TO FIX IT

- Add the `Content-Security-Policy` header to all server responses.
- Start with a restrictive policy: `Content-Security-Policy: default-src 'self'`
- Whitelist only trusted external domains explicitly.

6: Missing Anti-Clickjacking Header

Risk Level:	🟠 MEDIUM
Discovered By:	OWASP ZAP + Chrome DevTools
Affected:	All pages (Systemic)
OWASP:	A05:2021 — Security Misconfiguration

WHAT IS IT?

Clickjacking is an attack where a malicious website embeds the target site in an invisible iframe. The victim thinks they are clicking on the attacker's page but are actually clicking buttons on the real site — potentially authorizing transfers without knowing.

WHAT WAS FOUND?

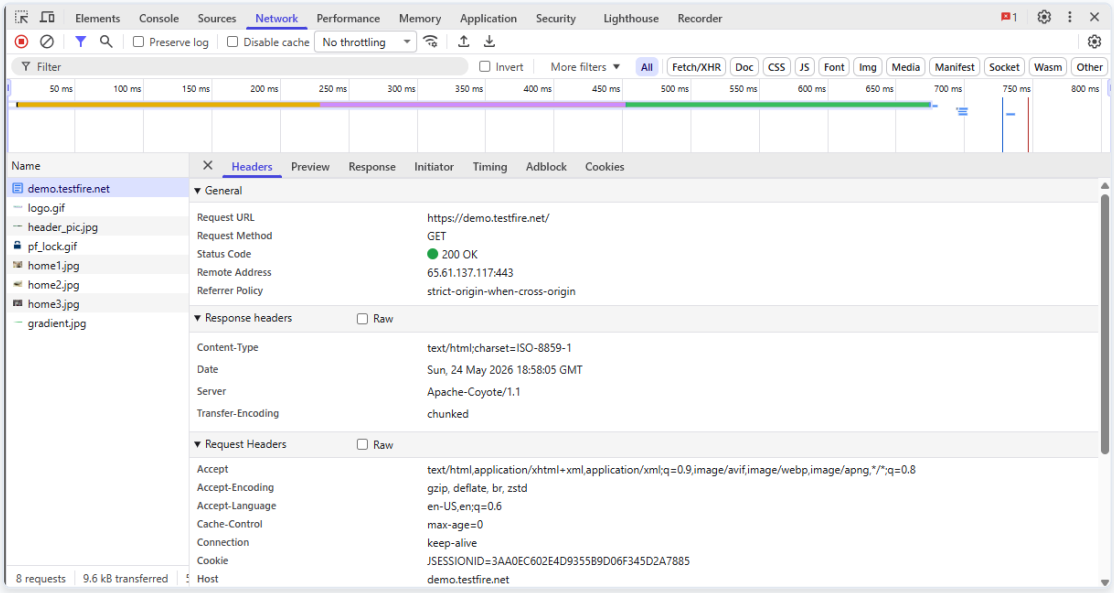
ZAP confirmed that neither `X-Frame-Options` nor a CSP `frame-ancestors` directive is set on any page. DevTools Response Headers confirmed this header is completely absent.

BUSINESS IMPACT

Any page on the banking application — including the login page and account management pages — can be embedded in an attacker's website, putting customers at risk of unintentional actions.

HOW TO FIX IT

- Add the following header to all server responses: `X-Frame-Options: DENY` (or SAMEORIGIN if you need internal framing).
- Configure this at the web server level so it applies to all pages automatically.



7: Source Code Disclosure — SQL Fragment

Risk Level:	🟠 MEDIUM
Discovered By:	OWASP ZAP
Affected URL:	/index.jsp?content=inside_trainee.htm
OWASP:	A05:2021 — Security Misconfiguration

WHAT IS IT?

Source code disclosure occurs when the web server accidentally serves fragments of server-side source code to the browser instead of processing it behind the scenes. This gives attackers insight into how the application handles data.

WHAT WAS FOUND?

ZAP detected a SQL fragment visible in a page response: `select "Management Trainee Program" from our` . This indicates part of a database query is being rendered as visible page content.

BUSINESS IMPACT

Exposed SQL fragments reveal database table names, query structures, and logic patterns. Attackers can use this information to craft much more targeted and effective SQL Injection attacks.

HOW TO FIX IT

- Audit all JSP pages that include dynamic content via query parameters.
- Ensure all server-side code is fully processed before being sent to the browser.
- Never render raw SQL or template variables as visible page content.

8: Subresource Integrity (SRI) Missing

Risk Level:	🟠 MEDIUM
Discovered By:	OWASP ZAP
OWASP:	A08:2021 — Software Integrity

WHAT IS IT?

Subresource Integrity (SRI) is a security feature that allows the browser to verify that scripts fetched from external servers have not been manipulated or tampered with by an attacker.

WHAT WAS FOUND?

The application loads JavaScript files from external sources without using the SRI attribute, meaning the browser blindly trusts whatever the external server provides.

BUSINESS IMPACT

If the external server hosting the script is compromised, attackers could alter the script to execute malicious JavaScript (XSS) directly within the context of the banking application, bypassing standard security controls.

HOW TO FIX IT

- Add the `integrity` and `crossorigin` attributes to all external `<script>` tags.
- This provides a cryptographic hash of the expected file content for the browser to verify.

9: Cookie Missing Security Attributes

Risk Level:	🟠 MEDIUM
Discovered By:	OWASP ZAP + Chrome DevTools
Cookie:	JSESSIONID
OWASP:	A07:2021 — Auth Failures

WHAT IS IT?

When a user logs in, they are given a session cookie (a digital ID badge). Security attributes like `HttpOnly`, `Secure`, and `SameSite` protect this cookie from being stolen by malicious scripts, transmitted over unencrypted networks, or used in cross-site attacks.

WHAT WAS FOUND?

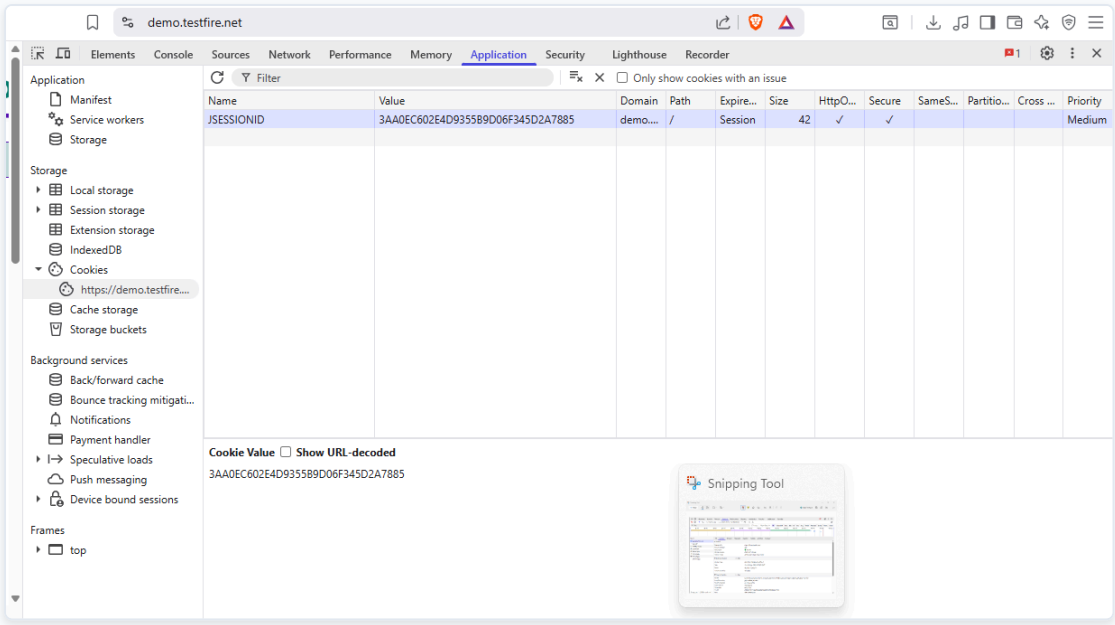
The DevTools Application tab shows the `JSESSIONID` session cookie. Not only is the `SameSite` attribute missing, but the critical security flags (`HttpOnly` and `Secure`) are also completely absent.

BUSINESS IMPACT

This leaves the session token entirely unprotected. Attackers can easily access the token via client-side JavaScript (XSS theft), intercept it over unencrypted networks, or leverage it for Cross-Site Request Forgery (CSRF) attacks.

HOW TO FIX IT

- Configure the server's `Set-Cookie` header to append `HttpOnly` and `Secure` flags to all session-related cookies.
- Set `SameSite=Strict` (or `Lax`) to prevent the cookie from being sent with cross-site requests.



10: Cross-Domain JS File Inclusion

Risk Level:	🟡 LOW
Discovered By:	OWASP ZAP
OWASP:	A05:2021 — Security Misconfiguration

WHAT IS IT?

The application is directly including and executing JavaScript files hosted on third-party domains that the organization does not own or control.

WHAT WAS FOUND?

ZAP identified that the site includes external scripts directly into the DOM without verification.

BUSINESS IMPACT

Relying on external domains introduces a supply-chain risk. If the third-party domain expires or is hijacked by a malicious actor, they gain immediate control over code running on the bank's site.

HOW TO FIX IT

- Whenever possible, download third-party scripts and host them locally on the bank's own infrastructure.

11: Server Version Disclosed

Risk Level:	🟡 LOW
Discovered By:	Nmap + Chrome DevTools
Header:	Server: Apache-Coyote/1.1
OWASP:	A05:2021 — Security Misconfiguration

WHAT IS IT?

When a web server responds to requests, it often includes a Server header that reveals exactly what software and version is running. This information is highly useful to attackers during reconnaissance.

WHAT WAS FOUND?

Both Nmap and DevTools Response Headers confirmed the server openly broadcasts its version: `Server: Apache-Coyote/1.1`.

BUSINESS IMPACT

Knowing the exact server software allows an attacker to search exploit databases (e.g., CVE) for known vulnerabilities targeting this specific version, reducing the effort required to compromise the server.

HOW TO FIX IT

- Configure Apache Tomcat to suppress or obscure the Server header in `server.xml` (e.g., set `server=" "`).

12: X-Content-Type-Options Missing

Risk Level:	🟡 LOW
Discovered By:	OWASP ZAP + Chrome DevTools
OWASP:	A05:2021 — Security Misconfiguration

WHAT IS IT?

The `X-Content-Type-Options` header prevents the browser from "MIME-sniffing" and trying to guess the content type of a file. Without it, browsers may ignore the server's instructions.

WHAT WAS FOUND?

ZAP and DevTools confirmed this header is absent from server responses.

BUSINESS IMPACT

An attacker could upload a malicious HTML/JavaScript file disguised as an image. The browser might "sniff" the content, realize it is HTML, and execute the script, leading to XSS.

HOW TO FIX IT

- Configure the web server to append `X-Content-Type-Options: nosniff` to all HTTP responses.

13: HTML Comments in Source Code

Risk Level:	🟡 LOW
Discovered By:	Chrome DevTools (Page Source)
Evidence:	<!-- BEGIN HEADER -->
OWASP:	A05:2021 — Security Misconfiguration

WHAT IS IT?

HTML comments are notes left by developers in the page source code. They are not displayed on the visual page but are fully visible to anyone who views the raw code.

WHAT WAS FOUND?

The page source contains developer comments like `<!-- BEGIN HEADER -->`.

BUSINESS IMPACT

While mostly harmless in this specific instance, leaving comments in live code can accidentally reveal sensitive internal structure, API endpoints, or disabled security features to attackers.

HOW TO FIX IT

- Implement an automated build process that strips all developer comments from the code before it is deployed to production.

```
<!-- BEGIN HEADER -->
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" >
```

14: Quirks Mode / Outdated DOCTYPE

Risk Level:	🟡 LOW
Discovered By:	Chrome DevTools
OWASP:	A05:2021 — Security Misconfiguration

WHAT IS IT?

The application does not specify a modern HTML5 DOCTYPE declaration at the top of its pages. This forces the browser to render the application in "Quirks Mode" (backwards-compatibility mode for older standards).

WHAT WAS FOUND?

DevTools inspection of the source code revealed the complete absence of `<!DOCTYPE html>`.

BUSINESS IMPACT

Quirks mode can cause unpredictable rendering issues and may inadvertently disable modern browser security mechanisms, making the application slightly more susceptible to client-side quirks.

HOW TO FIX IT

- Ensure that the very first line of every HTML/JSP response is the standard HTML5 declaration: `<!DOCTYPE html>`.

CONCLUSION

The vulnerability assessment of `demo.testfire.net` (Altoro Mutual) identified 14 security findings across High, Medium, and Low severity levels.

The most critical concern is the confirmed SQL Injection vulnerability on the login form — this requires immediate remediation as it poses a direct risk of unauthorized database access. The Slowloris DoS exposure and weak Diffie-Hellman TLS configuration represent additional high-priority items that could affect service availability and data confidentiality.

At the systemic level, the complete absence of Content Security Policy, X-Frame-Options, and Anti-CSRF tokens across all pages represents a foundational security gap that can be

resolved through server-level configuration changes rather than individual code fixes.

Remediation Priority Summary:

IMMEDIATE (CRITICAL/HIGH RISK):

- Patch SQL Injection in login form
- Configure Slowloris DoS protections
- Upgrade TLS DH key exchange to 2048-bit minimum

SHORT-TERM (MEDIUM RISK):

- Implement Content Security Policy headers
- Add X-Frame-Options to all responses
- Add Anti-CSRF tokens to all forms
- Configure Secure, HttpOnly, and SameSite attributes on session cookies

ONGOING (LOW RISK):

- Suppress Server version header
- Remove HTML developer comments from production code

Regular security assessments and developer security training are recommended as ongoing practices to maintain a strong security posture.

Assessed by: Sandile Theron Lubisi

Date: May 25, 2026

Internship: Future Interns — Cyber Security Track